Apuntes – MVVM y Repository

Desarrollo de Aplicaciones I · UADE · Guía de Ejercicios: MVVM y Repository (v1.0)

Proyecto: `MvvmLab` · Felipe Forte

Organización del proyecto en carpetas

El código está organizado por **capas**, con la misma estructura que el repositorio de referencia *PokedexAcademy-MVVM-Clean*. Cada carpeta es un *paquete* de Kotlin (`com.example.mvvmlab.presentation.nota`, etc.). En cada rama aparecen solo las carpetas que ese ejercicio usa.

```
app/src/main/java/com/example/mvvmlab/
 ├── MainActivity.kt          # punto de entrada: aloja la UI
 ├── MvvmLabApp.kt            # Application propia (desde el Ej 11)
 ├── core/
 │    ├── model/              # tipos comunes a varias pantallas (EstadoCarga)
 │    ├── navigation/         # rutas de navegación (desde el Ej 12)
 │    ├── theme/              # tema de Compose (Color, Theme, Type)
 │    └── util/               # utilidades (Fechas)
 ├── data/                    # CÓMO se guardan los datos
 │    ├── local/              # Room: Entity, DAO, Database (Ej 14–15)
 │    ├── mapper/             # Entity ⇄ modelo de dominio
 │    └── repository/         # implementaciones de los contratos (memoria, archivos, Room)
 ├── domain/                  # QUÉ es la app: Kotlin puro, sin Android
 │    ├── model/              # Nota, Registro, ResultadoLectura
 │    ├── repository/         # contratos (interfaces) de los Repository
 │    └── validacion/         # reglas de negocio (ValidadorNota, ValidadorRegistro)
 ├── presentation/            # MVVM: una carpeta por pantalla (UiState + ViewModel + Pantalla)
 └── di/                      # quién crea las dependencias (ContenedorApp, Ej 11)

app/src/test/java/com/example/mvvmlab/
 ├── domain/validacion/       # tests de reglas
 ├── fake/                    # Repository falsos para tests
 ├── presentation/            # tests de ViewModels
 └── util/                    # MainDispatcherRule
```

Capa	Regla
`domain/`	No importa nada de Android, Compose, Room ni archivos. Define los datos (`model`), los contratos (`repository`) y las reglas (`validacion`).
`data/`	Implementa los contratos de `domain`. Es la única capa que conoce archivos, SQL o Room.
`presentation/`	ViewModels y pantallas. Solo habla con los contratos de `domain`, nunca con `data` directamente.
`di/`	Conecta todo: elige qué implementación de `data` recibe cada ViewModel.

¿Y los *use cases* del repositorio de referencia?

El repositorio de referencia tiene además `domain/usecase/`. Acá no se agregaron: la guía pide no crear capas "porque sí", y en estos ejercicios cada use case solo llamaría a un método del Repository. La teoría (diapositiva 4) aclara que la capa de dominio es opcional.

Ejercicio 1 — ¿Quién está haciendo demasiado?

Consigna: tomar una pantalla donde todo está mezclado, marcar dónde se dibuja la interfaz, dónde se validan los datos, dónde se modifica la lista y dónde se guarda, y ubicar cada responsabilidad en *View*, *ViewModel* o *Model/Repository*. Todavía **no** se cambia la implementación.

Conceptos base

MVVM = separar en 3 responsabilidades

- View (Compose) **dibuja** el estado y **avisa** lo que hace el usuario (clicks, texto). No decide nada.
- ViewModel **recibe** la acción del usuario, **decide** (valida, coordina) y **expone** el estado de pantalla.
- Model / Repository **obtiene y guarda** los datos (memoria, archivo, Room, API). No sabe nada de pantallas.

UI = f(estado): la pantalla es una función del estado.

¿Cómo decido dónde va cada cosa?

El criterio **no** es "¿declara una variable?" — hay variables en las tres capas. Las preguntas útiles son:

1. **¿Esto dibuja algo?** → View. *(Test: si cambio Compose por una web, ¿tengo que reescribir esta línea? Sí → View.)*
2. **¿Es una regla que valdría igual en cualquier interfaz?** → ViewModel.
3. **¿Este dato tiene que sobrevivir si cierro la app?** Sí → dato real → Repository. No, solo importa mientras uso la pantalla → estado de pantalla → ViewModel.

Ejemplo: en WhatsApp, el mensaje que estás escribiendo (borrador) es de la pantalla; los mensajes ya enviados se guardan.

El ViewModel decide QUÉ mostrar. La View decide CÓMO mostrarlo.

"Hay un error: tarea vacía" → ViewModel. "El error va en rojo, debajo del botón" → View.

Código analizado (pantalla “mal hecha a propósito”)

```
@Composable
fun PantallaTareas(context: Context, modifier: Modifier = Modifier) {
    var texto by remember { mutableStateOf("") }                    // (A)
    var tareas by remember { mutableStateOf(listOf<String>()) }     // (B)
    var error by remember { mutableStateOf<String?>(null) }         // (C)

    Column(modifier.padding(16.dp)) {                                // (D)
        Text("Mis tareas (${tareas.size})")
        OutlinedTextField(value = texto, onValueChange = { texto = it })

        Button(onClick = {
            if (texto.isBlank()) {                                   // (E)
                error = "La tarea no puede estar vacía"
            } else if (tareas.contains(texto)) {                     // (F)
                error = "Esa tarea ya existe"
            } else {
                tareas = tareas + texto                              // (G)
                File(context.filesDir, "tareas.txt")                 // (H)
                    .writeText(tareas.joinToString("\n"))
                texto = ""                                           // (I)
                error = null
            }
        }) { Text("Agregar") }

        error?.let { Text(it, color = Color.Red) }                   // (J)
        LazyColumn { items(tareas) { Text(it) } }                    // (K)
    }
}
```

Resultado: tabla de responsabilidades (entregable)

Letra	Código	Qué hace	Dónde va
(A)	`var texto`	Borrador que escribe el usuario; no sobrevive al cierre	ViewModel
(B)	`var tareas`	Lista real de tareas; tiene que sobrevivir al cierre	Repository
(C)	`var error`	Mensaje de error actual (dato del estado de pantalla)	ViewModel
(D)	`Column`, `Text`, `OutlinedTextField`	Dibuja la pantalla	View
(E)	`texto.isBlank()`	Regla: no se agregan tareas vacías	ViewModel
(F)	`tareas.contains(texto)`	Regla: no se agregan tareas repetidas	ViewModel
(G)	`tareas = tareas + texto`	Agrega la tarea a la lista (el dueño del dato la modifica)	Repository
(H)	`File(...).writeText(...)`	Guarda las tareas en un archivo	Repository
(I)	`texto = ""`	Limpia el campo solo si la tarea se agregó bien (decisión sobre un dato del ViewModel)	ViewModel
(J)	`error?.let { Text(it, Color.Red) }`	Muestra el error en rojo (el error es del VM; dibujarlo es de la View)	View
(K)	`LazyColumn`	Muestra la lista en pantalla	View

Errores frecuentes (los que cometí y corregí)

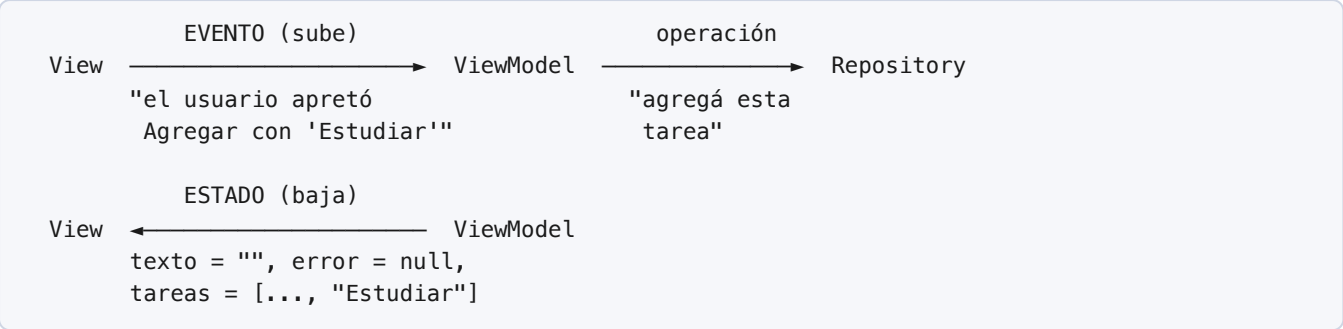
- Pensar que (A), (B) y (C) van al Repository "porque declaran variables". Solo (B) es un dato que debe persistir.
- Poner (I) en la View: `texto` es del ViewModel, y quien es dueño de un dato es quien lo modifica.
- Poner (J) en el ViewModel: el *error* es del ViewModel, pero *mostrarlo* con `Text` y `Color.Red` es de la View.

La cadena completa de "Agregar"

1. View el usuario aprieta *Agregar* → avisa al ViewModel.
2. ViewModel aplica las reglas (E) y (F). Si pasan → le pide al Repository que agregue; limpia el campo (I).
3. Repository agrega la tarea (G) y la guarda en el archivo (H).

Repository = **dueño del dato** · ViewModel = **intermediario** (lo pide y lo prepara) · View = **solo lo muestra**. La lista "verdadera" vive en un solo lugar: **fuente de verdad única**.

Desafío: las dos flechas (UDF – flujo unidireccional de datos)



> **Evento vs. estado**
>
> - **Evento** = algo que *pasó* (un click, una tecla). Es puntual.
> - **Estado** = cómo *está* la pantalla ahora. Se puede consultar en cualquier momento.
> - **UDF:** los eventos suben y el estado baja. La View nunca modifica el estado por su cuenta: solo avisa y redibuja.

Preguntas de la guía

1. ¿Qué trabajo tendría que seguir funcionando aunque mañana cambiemos Compose por otra interfaz?
Las **reglas** (E, F: no vacía, no repetida) y el **manejo de datos** (G, H: agregar y guardar). Es decir, todo lo que va en ViewModel y Repository.

2. ¿Qué datos existen sólo mientras se está escribiendo un formulario?
`texto` (el borrador) y `error` (el mensaje actual). Son estado de pantalla y no se guardan en el archivo.

> **Señal de alarma: `Context` en un composable**
>
> `PantallaTareas` recibe un `Context` solo para poder escribir el archivo (H). La guía advierte no aceptar soluciones que "llevan Context a todos los composables" o "abren archivos desde un botón". Es una pista de que el acceso a archivos está en el lugar equivocado: debería estar en el Repository.

Kotlin / Compose que apareció en el camino

Elemento	Qué es
`Unresolved reference: X`	Kotlin no sabe qué es X: falta el `import`, está mal escrito o la función/clase no existe.
`var x by remember { mutableStateOf(...) }`	Estado observable que sobrevive a la recomposición. El `by` necesita los imports `getValue` / `setValue`.
`LocalContext.current`	Forma de obtener el `Context` de Android dentro de un composable.
`modifier: Modifier = Modifier`	Convención: permite que quien llama le pase padding, tamaño, etc. (por ejemplo el `innerPadding` del Scaffold).
`@Preview` / `@Composable`	Anotaciones: "etiquetas" que siempre se aplican a una función. Solas, sin función debajo, no compilan.